Guía de Laboratorio Semana 3

Análisis de Algoritmos y Estructuras de Datos

Carrera Ingeniería de Ejecución en Informática

2026-2

Objetivos del laboratorio

Escribir, compilar y ejecutar programas en lenguaje C sobre Ubuntu usando **punteros y pasada por referencia**, **matrices**, **archivos** y **registros (struct)**

Herramientas que debe utilizar:

- **Terminal Ubuntu**
- **Editor de Texto**
- **Compilador GCC ejecutado desde la terminal.**

Dentro de Ubuntu, abra una terminal y cree el directorio **Labsem3**. Ingrese al directorio que acaba de crear. Recuerde los comandos que puede usar:

COMANDO	FUNCIÓN
`pwd`	Muestra el directorio actual.
`mkdir -p` **Labsem3**	Crea un nuevo directorio dentro del directorio actual. Si ya existe no lo crea nuevamente.
`ls`	Muestra los archivos y carpetas del directorio actual.
`ls -l`	Muestra los archivos con información detallada.
`ls -a`	Muestra también los archivos ocultos, cuyos nombres comienzan con un punto.
`cd ~/`**Labsem3**	Ingresa al directorio de trabajo **Labsem3** ubicado sobre mi directorio personal.
`cd ..`	Desde el directorio actual, regresa al directorio anterior.
`cd ~`	Va al directorio personal del usuario.
`clear`	Limpia la pantalla de la terminal.
`rm archivo`	Elimina un archivo
`herramienta --version`	Permite saber si un comando (o herramienta de línea de comando) está instalado. Ejemplo: si se desea saber si el compilador **gcc** está instalado se ejecuta: **gcc --version**

Actividad 1: Implemente en C un algoritmo llamado **invertir** que, dado un arreglo A de n enteros, intercambie los elementos de cada par de posiciones consecutivas: A[0] con A[1], A[2] con A[3], y así sucesivamente. Utilice una función swap con parámetros por referencia. Si n es impar, el último elemento debe permanecer sin cambios. Los datos del arreglo de entrada al algoritmo se encuentran guardados en un archivo.
Ejemplo:
Después de leer los datos del archivo, la entrada al algoritmo es por ejemplo el arreglo:
A = [3, 7, 2, 8, 5, 9]
y la salida debe ser el mismo arreglo:
A = [7, 3, 8, 2, 9, 5]

Ayuda de implementación:

/ * La función swap conocida es la siguiente. **¿Cómo quedaría esta función swap si los valores de A[i] y A[i+1] se pasan por referencia?** */

```
void swap(int a, int b) {
   int aux = a;
   a = b;
   b = aux;
}
/* esta es la implementación solicitada */
void invertir (int *A, int n){
    //aquí (en alguna parte) debiera llamar a la función SWAP
}
```

/* Use el main siguiente para obtener el arreglo A que le servirá como entrada a su algoritmo y poder probar su implementación (revise la ppt-lab-semana3) para completar la lectura de los datos del archivo: */

```
int main(int argc,  char *argv[]){
   int n=0;
   if (argc != 2)
         return 1;
   int *A = leer_datos_Fil(argv[1], &n);
   if ( A == NULL )
         return 1;
   for (int i = 0; i < n; i++)
         printf("%d ", A[i]);

   // aquí ejecute el algoritmo solicitado:
   invertir(A, n)
   // aquí muestre el resultado de A después de invertir:

   free(A);
   return 0;
}
```

Genere y ejecute su programa con:

gcc invertir.c -o invertir.exe
./invertir.exe dataA.txt

Actividad 2: Implemente en C un algoritmo llamado **posCercano_al_prom** que dado un arreglo A de n elementos, retorne la posición del elemento cuyo valor sea el más cercano al promedio de los elementos del arreglo **A**. Los datos del arreglo de entrada al algoritmo se encuentran guardados en un archivo.

Ejemplo:

Si los datos que se guardan en el archivo para la entrada A es:

dataA.txt

```
10
7
3
6
4
21
9
55
45
100
20
```

El promedio de estos valores es 27. Luego el elemento más cercano a 27 es el 21 que ocupa la posición 4 del arreglo A. La función debiera retornar esta posición.

Ayuda de implementación:

```
#include<stdio.h>
#include<stdlib.h>
int posCercana_Aprom(int A, int n){
  ...
}
/ *Use la misma función main anterior para leer los datos del archivo y guadarlos en un arreglo A
(mire el primer la lectura de archivos (guardar en arreglo) de la ppt-lal-semana3) */

int main(int argc,  char *argv[]){

   int n=0;
   if (argc != 2)
         return 1;
   int *A = leer_datos_Fil(argv[1], &n);
   if ( A == NULL )
         return 1;
   for (int i = 0; i < n; i++)
         printf("%d ", A[i]);

   // aquí ejecute el algoritmo solicitado:
   int pos = posCercana_Aprom(A, n)
   // muestre pos con printf (el resultado según el ejemplo debiera ser4)
   ...
   free(A);
   return 0;
}
```

Genere y ejecute su programa con:

gcc posCercano_al_prom .c -o posCercano_al_prom .exe

./posCercano_al_prom .exe dataA.txt

Actividad 3: Implemente en C un algoritmo llamado **matriz_conpares** que dada una matriz M de enteros de tamaño nxm, retorne la matriz solo con los elementos pares, en caso de existir un elemento impar, este debe ser cambiado a cero.

Ayuda de implementación:

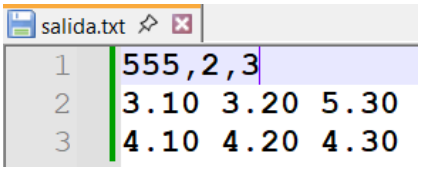
Use la misma función main para leer los datos del archivo y guadarlos en una matriz M (mire la lectura de archivos **guardar en matriz** de la ppt-lab-semana3)

Genere y ejecute su programa con:

gcc matriz_conpares.c -o matriz_conpares.exe
./matriz_conpares.exe dataM.txt

Actividad 4: Termine de implementar el problema "Registro con struct y tipo de datos creado por usuario llamado **notasEDA**" de la ppt-lab-semana3. El problema es el siguiente: Cree un registro para guardar las calificaciones de un estudiante de la asignatura EDA. El registro debe tener el id del estudiante, sus notas de PEP y de controles. Asuma que el número de notas PEPs y de controles es la misma. Cree el tipo de datos **notasEDA** para este registro. Su programa C debe hacer los siguiente:

a) Implemente una función que ingrese las notas (con scanf) de un id alumno dado y las guarde en una variable de tipo **notasEDA**. Los arreglos de las notas definidos en el struct deben crearse con tamaño dinámico (usando malloc).
Para no perder las notas ingresadas, guárdalas en un archivo donde cada fila corresponde a un tipo de nota y las notas separadas por espacio. La primera fila debe tener el id del estudiante, el número de filas (tipos de notas) y el número de notas para cada tipo. **El archivo resultante debe ser algo como esto:**



b) Dada una ***estructura de registro usada a través de una variable de tipo*** **notasEDA** que almacena las notas de un estudiante obtenidas del archivo de salida anterior, implemente una nueva función que entregue la peor nota de Peps con su promedio y la mejor nota de controles con su promedio.